```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract Bank {
   mapping(address => uint256) public balances;

   // 1. Added 'payable' and switched from a manual parameter to 'msg.value'
   function depositMoney() public payable {
      require(msg.value > 0, "Amount must be greater than zero");
      balances[msg.sender] += msg.value;
   }
  // 2. Added the actual transfer logic to send Ether back to MetaMask
   function withdrawMoney(uint256 amount) public {
      require(amount > 0, "Amount must be greater than zero");
      require(balances[msg.sender] >= amount, "Insufficient balance");

      // Deduct from balance first to keep it secure
      balances[msg.sender] -= amount;
     // This line actually transfers the real test Ether back to your MetaMask wallet
      (bool success, ) = msg.sender.call{value: amount}("");
      require(success, "Transfer failed");
   }

   function showBalance() public view returns (uint256) {
      return balances[msg.sender];
   }
}
```

Using **1 Gwei** means each deposit only costs 0.000000001 ETH. This is so microscopic that your 0.05 ETH faucet balance will last practically forever.**Step 1: The Micro-Deposit**

1. Go to the top **Value** field in Remix.
2. Change the dropdown unit from ether to **gwei**.
3. Type **1** into the box.
4. Click the orange **depositMoney** button and confirm in MetaMask.

Step 2: Read the Output

1. Clear the top value box back to **0**.
2. Click the blue **showBalance** button.
3. **The Output:** Directly underneath, it will display exactly:
   uint256: 1000000000 *(exactly **9 zeros** after the 1)*.

Step 3: The Micro-Withdrawal

1. Go to the input field next to the orange **withdrawMoney** button.
2. Type **1000000000** *(9 zeros)* to take out the whole amount, or **500000000** *(8 zeros)* to take out exactly half.
3. Click **withdrawMoney** and confirm in MetaMask

because smart contracts handle value natively in Wei, the contract logic performs the exact same state updates and access controls whether we process 1 Gwei or 100 Ether. I am intentionally using 1 Gwei to optimize gas consumption and preserve our testnet faucet resources, demonstrating efficient block resource management."

UnitName	***How much Ether is it?***	***What is it used for? (Real-world explanation)***
Ether (ETH)	*1 ETH*	*Used for **large transactions** and listing prices (like buying an NFT or checking your main wallet balance).*
Finney	*0.001 ETH*	*Used as a **mid-range unit**, convenient for micro-transactions or small lab test deposits.*
Gwei (Gigawei)	*0.000000001 ETH*	*Used exclusively to measure **Gas Fees**. When network traffic is high, gas prices go up in Gwei.*
Wei	*0.000000000000000001 ETH*	*The **absolute smallest unit** of Ethereum. This is the **only** unit the smart contract code actually uses to do math.*

Development Method	**Language for Scripts/Tests**	**Best Used For**	**Speed / Performance**
Remix IDE	JavaScript / GUI	Quick lab practicals, learning basics, prototyping concepts	Slow (Browser-bound)
Hardhat	JavaScript / TypeScript	Production dApps, detailed step-by-step transaction debugging	Medium
Foundry	**Solidity**	High-performance enterprise suites, advanced mathematical optimization	Extremely Fast (Rust engine)
Brownie	Python	Web3 projects integrated with AI, automation data, or machine learning	Medium

- Bitcoin is a decentralized digital currency system. Its primary purpose is to act as digital gold or a peer-to-peer payment network. Its script is intentionally limited to simple transactions (Sending money from A to B).
- Ethereum is a decentralized software platform. While it has its own currency (Ether), its main purpose is to execute Smart Contracts.

a blockchain is a decentralized, digital ledger that securely records transactions across a network of computers, making it impossible to alter, hack, or cheat the system. Why it's secure: Data is packed into Blocks. Each block is chained to the previous one using a unique digital fingerprint called a Hash. If a hacker tries to change block 2, the fingerprint changes, breaking the whole chain, and the network immediately rejects it.

Type	How it Works	Example
Public	**Completely open.** Anyone in the world can join, read data, write transactions, and audit the code.	Bitcoin, Ethereum, Sepolia Testnet
Private	**Strictly controlled.** Only a single organization owns it and decides who can join or read data.	A bank tracking internal money transfers
Consortium	**Shared control.** Owned by a small group of pre-approved companies or institutions instead of just one.	Multiple banks managing international shipping logistics
Hybrid	**Mixed access.** Controlled by one company, but keeps a public section open for customer verification.	A medical system tracking drug shipments privately, but letting patients scan a QR code publicly

- **Supply Chain Tracking (e.g., IBM Food Trust / Walmart):** Companies use blockchain to track food or products from the farm to the store shelf. **Digital Identity & Voting:**Governments use blockchain to create tamper-proof digital IDs.
- **Real Estate & Land Registry:**In many developing countries, corrupt officials can alter land deeds. Putting property deeds on a public blockchain makes land ownership permanent, unchangeable, and completely transparent.
- **Decentralized Finance (DeFi) & Insurance:** Smart contracts (like the Bank contract you built!) can automatically pay out flight insurance to travelers if a commercial airline database updates to show their flight was delayed by more than 2 hours—completely eliminating paperwork.

Gas fees are paid to network validators to process transactions and compute code changes on the blockchain. **Deposits and withdrawals modify the blockchain state storage**, which costs gas. **Checking the balance is a view function**—it only reads data without changing it, so it executes locally for free.

line (bool success,) = msg.sender.call{value: amount}(""); do?Answer: *"This is the modern, recommended way in Solidity to send native Ether to an address. It transfers the specified amount of Wei back to the caller's wallet and returns a boolean value (true or false) to indicate if the transfer succeeded.*
ERC-20 is a set of rules that defines how to create a standard fungible token on the Ethereum blockchain."
Fungible means interchangeable: *Just like one 10-rupee note or one dollar bill is exactly identical to and interchangeable with another, ERC-20 tokens are identical in value.*
The Analogy: *Think of ERC-20 like a* ***universal phone charger recipe****. Because all manufacturers follow the same recipe (Type-C), their chargers fit every phone. Similarly, because tokens follow the ERC-20 recipe, any crypto wallet (like MetaMask) or exchange can automatically hold and transfer them without writing new code.*
The Core Rules (Functions): *It forces every token creator to include 6 basic functions:*

1. *totalSupply() – How many tokens exist in total.*
2. *balanceOf() – Checks how many tokens a specific wallet holds.*
3. *transfer() – Sends tokens from your wallet to a friend.*
4. *transferFrom() – Allows a smart contract to move tokens on your behalf.*
5. *approve() – Gives a smart contract permission to spend your tokens.*
6. *allowance() – Checks how many tokens a contract is allowed to spend*

appli: ***Tether (USDT) & USD Coin (USDC):****Stablecoins whose values are fixed to match the US Dollar.*
Chainlink (LINK): *A token used to pay for data services on the Ethereum network.*
Uniswap (UNI): *A governance token used to vote on updates for a popular crypto exchange.*

#student database

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.20;

contract StudentData {

  // Structure to store student details

  struct Student {

    uint256 rollNo;

    string name;

    uint256 marks;
```

```
    }
   // Array to store students
    Student[] public students;
    // Add a student
    function addStudent(
        uint256 _rollNo,
        string memory _name,
        uint256 _marks
    ) public {
        students.push(Student(_rollNo, _name, _marks));
    }
    // Get student details using array index
    function getStudent(uint256 index)
        public
        view
        returns (uint256, string memory, uint256)
    {
        require(index < students.length, "Invalid student index");

        Student memory s = students[index];
        return (s.rollNo, s.name, s.marks);
    }
    // Get total number of students
    function getStudentCount() public view returns (uint256) {
        return students.length;
    }
    // Fallback function
    fallback() external payable {
    }
    // Receive ETH without calldata
    receive() external payable {
    }
}
```

After the student process..for fallback you have to expand the low level transaction:

1. Scroll down to your deployed contract sidebar. 2.Collapse the regular functions (addStudent, getStudent) so you only see the section at the bottom called **Low level interaction**.3.Look for the **Calldata** input field.4.Type any random hex data into that field, such as **0x1234**.
   (Reason: This simulates a broken or unknown function call that will confuse the contract).

Step 2: Set the Value (Optional) Since your fallback is marked as payable, you can send money with it.**2.**Go to the top **Value** field, select **gwei**, and type **10**.

Step 3: Click Transact

1. Click the blue **Transact** button located directly inside that bottom **Low level interaction** box.2.**MetaMask** will pop up asking you to sign a transaction to your contract address. Click **Confirm**.

What fallback() does: It acts as a safety net. If a user or a script calls a function name that does not exist in my contract (like sending 0x1234), the contract doesn't crash; it automatically redirects the call into the fallback() function.**Why receive() is different:** If a user sends raw Ether to the contract **without typing any calldata** (empty box), the contract routes it to receive(). But because we typed 0x1234 into the Calldata box, the contract was forced to run fallback() instead.

In a professional real-world system, a student database smart contract acts as an **Automated Tuition & Fees Collection Desk** for a university. Imagine a university student named Rahul. The college sets up a smart contract to track his marks, attendance, and roll number. But the college also says: *"If you send raw money directly to this contract address, it will automatically register as your college tuition payment."*

- **The Benefit:** Rahul doesn't need to navigate a complicated university portal dashboard. He can simply open his MetaMask wallet app on his phone, paste the StudentData contract address, type 0.01 ETH, and hit send.
- **The Mechanism:** Because your contract has receive() external payable {} or fallback(), the contract successfully absorbs his tuition fee payment instead of rejecting his money.

A fallback function is a special function in Solidity that executes when a smart contract receives a call with data that does not match any existing function

Gas is the unit used to measure the computational work required to execute an operation on the Ethereum blockchain. Think of gas like the amount of fuel a vehicle needs to travel. More computational work generally requires more gas.(How much computational work was done)

A transaction fee (Total amount paid) is the amount of ETH paid to process a transaction on the Ethereum network.Transaction Fee = Gas Used × Gas Price if a transaction uses 50,000 gas and the gas price is 2 Gwei:50000*2Gwei=0.0001ETH

Gas Price is the amount paid for each unit of gas.(Cost per unit of computational work) It is commonly measured in Gwei.For example, if the gas price is 2 Gwei, each unit of gas costs 2 Gwei. One Gwei equals =0.000000001 ETH.

PRAC 1& 2:

Ethereum is a public, open-source blockchain platform that acts as a global, decentralized computer.Unlike Bitcoin (which is just digital cash), Ethereum introduces **Smart Contracts**—self-executing digital agreements that run automatically without middlemen.It serves as the foundation for building Decentralized Applications (dApps), NFTs, and custom crypto tokens.

The Concept of Ether (ETH)

Ether (ETH) is the native cryptocurrency of the Ethereum blockchain.It serves as the "fuel" or **Gas** needed to run operations, execute smart contracts, or transfer assets on the network. Every time you do something on Ethereum, you pay a small fee in Ether to compensate the network validators for their computing power.

MetaMask is a highly popular crypto wallet that acts as a bridge between your web browser and the Ethereum blockchain.

User-Friendly: It operates as a simple browser extension or mobile app, making blockchain easy to interact with.

Gateway to dApps: It lets users instantly connect to decentralized apps, crypto games, and NFT marketplaces with one click.

Self-Custodial: You hold your own private keys (Secret Recovery Phrase), meaning you have 100% control over your funds without banks.

Token Swapping: It has a built-in exchange feature allowing users to swap one ERC-20 token for another instantly.

A **Blockchain Wallet** is a digital tool (software or hardware) that allows users to interact with a blockchain network. **How it works:** A wallet does *not* actually store physical coins. Instead, it securely holds two cryptographic keys:**Public Key:** Your wallet address (like a bank account number) that you share with others to receive funds.

Private Key: Your secret password (like a digital signature) used to approve transactions. **If you lose this, you lose your funds.**

Main Usages: Sending and receiving cryptocurrencies.Checking account balances.Signing transactions to interact with smart contracts.

Etherscan is a public block explorer for the Ethereum network. It works like a search engine for the blockchain, allowing anyone to transparently view real-time data.When you search an Ethereum address or transaction on Etherscan, you can observe:

Ether Balance: The exact amount of ETH currently held in a specific wallet.**ETH Value:** The current real-time market value of that Ether in US Dollars.**Transaction History:** A complete public ledger showing every past incoming and outgoing transaction, including timestamps and sender/receiver addresses.**Gas Fees Paid:** Exactly how much Ether was spent on network fees for a specific transaction.

Case Study 1: Storing Simple Values (Bank.sol)

Let's look at what happens when you modify a basic storage number (like depositing money).

Operation: depositMoney() **Gas Used:** ~45,000 gas **Live Gas Price during your test:** ~100 Gwei (100×10^{-9} ETH)

Calculation:
$45,000\times 0.0000001\text{\ ETH}=\mathbf{0.0045}\text{\ SepoliaETH}$

Study Conclusion: Modifying a simple integer or tracking an account mapping uses low computational energy. Therefore, it spends a very small fraction of Ether.

Let's analyze the exact real-world transaction log you provided earlier for your fallback function.

- **Operation:** StudentData.(fallback)
- **Your Real Gas Used: 22,280 gas**
- **Why was it this number?** The baseline cost to send *any* transaction on Ethereum is a fixed minimum of 21,000 gas. Because your fallback function block {} was completely empty, the code execution itself only cost an extra 1,280 gas (22,280 - 21,000).
- **Study Conclusion:** Empty fallback functions are highly optimized. They spend the absolute bare minimum amount of Ether possible because they do not write any new data to the blockchain storage layer.

---

4. Case Study 3: Reading Data (showBalance())

- **Operation:** showBalance() or getStudent()
- **Gas Used: 0 gas**
- **Ether Spent: 0 ETH**
- **Study Conclusion:** Functions marked with the **view** or **pure** keyword do not modify anything on the blockchain ledger. Because they only read data, they can run locally inside your browser without broadcast to network validators. Hence, they spend zero Ether

---

Wallets are primarily divided into two main operational categories:**Hot Wallets (Connected to Internet):**

Software/Extension Wallets: Browser extensions or mobile apps used for daily transactions. (e.g., **MetaMask, Trust Wallet**).**Exchange Wallets:** Wallets managed by centralized exchanges. (e.g., **Binance, Coinbase**).

Cold Wallets (Offline/Most Secure):

Hardware Wallets: Physical USB-like devices that store your private keys completely offline away from hackers. (e.g., **Ledger, Trezor**).**Paper Wallets:** Printing your public address and private key onto a physical piece of paper as a QR code.

A cryptocurrency is a decentralized digital asset secured by cryptography, operating natively on a blockchain network without relying on any central bank or government entity."

Example 1: Bitcoin (BTC): The very first cryptocurrency. It acts primarily as **Digital Gold** or a scarce peer-to-peer store of value. It runs on its own isolated payment ledger.

Example 2: Ether (ETH): The native token of the **Ethereum network**. It acts like **Digital Fuel (Gas)**. You use it to pay network validators to run your code logic, execute your Bank deposits, or update records inside your StudentData system.

Advantages of Wallet Creation in Blockchain

Complete Ownership (Self-Custody): No bank or third party can freeze your account, block your transfers, or deny you access to your money.

Permissionless Access: Anyone with an internet connection can create a wallet in 30 seconds. No KYC documents, background checks, or credit ratings are required.

Global, Direct Transactions: You can transfer funds directly to any other wallet address across the world instantly without waiting for banking holidays or clearing clearing delays.

Single Unified Login: A web3 wallet acts as your global identity, allowing you to log into thousands of different dApps and developer portals (like Remix IDE) with a single click.

Unique Identification is Given to a Wallet?

Every wallet is assigned two unique cryptographic strings of alphanumeric characters:

- **The Public Address (Your Account Number):**
    - **Format:** On Ethereum, it is a 42-character hexadecimal string starting with **0x** (e.g., your test account 0x71c5...6B82).
    - **Use Case:** You can share this openly with anyone so they can send you funds, view your transaction history, or interact with your contract.
- **The Private Key (Your Digital Signature):**
    - **Format:** A unique 64-character hexadecimal string paired strictly with your public address.
    - **Use Case:** This is hidden inside MetaMask. It is used automatically to sign and approve transactions (like when you click "Confirm" to pay gas fees). It must never be shared.
